JODHPUR INSTITUTE OF ENGINEERING AND TECHNOLOGY

Jodhpur, Rajasthan

PROJECT REPORT

On

Secure File Encryption System

Submitted in partial fulfillment for the subject
Security Lab [6CS4-23]

Submitted By:

Neeraj Kumhar
Roll No: 24EJICS201
Registration No: JIET24CSD003
Semester: 6th (3rd Year)
Branch: Computer Science Engineering
Section: A3

Submitted To:

Ms. Sapna Patel
Assistant Professor

Mrs. Anju Jangid
HOD, CSE Department

Dr. Naresh Purohit
HOD, CSE Department

Department of Computer Science Engineering
JIET - Jodhpur Institute of Engineering & Technology
Academic Session 2025-26

Acknowledgement

I would like to express my sincere gratitude and appreciation to my subject teacher **Ms. Sapna Patel**, Assistant Professor, Department of Computer Science Engineering, Jodhpur Institute of Engineering and Technology, for her valuable guidance, continuous support, and constructive suggestions throughout the development of this project. Her encouragement and technical insights helped me understand the concepts of cybersecurity and encryption in a practical manner.

I would also like to extend my heartfelt thanks to **Mrs. Anju Jangid, Head of the Department (CSE)** and **Dr. Naresh Purohit, Head of the Department (CSE)**, for providing the necessary facilities, resources, and academic environment that made the successful completion of this project possible.

I would like to thank all the faculty members of the Department of Computer Science Engineering for their support and for sharing their knowledge during the course of my academic studies.

I am also grateful to my classmates and friends for their cooperation, valuable discussions, and encouragement during the development and testing of this project.

Finally, I would like to express my deepest gratitude to my family members for their constant motivation, moral support, and encouragement throughout my academic journey, which helped me successfully complete this project.

Neeraj Kumhar
Roll No: 24EJICS201
Registration No: JIET24CSD003

Abstract

In the modern digital era, a large amount of information is stored and transferred in digital form. As a result, protecting sensitive data from unauthorized access has become one of the most critical challenges in the field of information security. Encryption plays a vital role in ensuring data confidentiality by converting readable information into an unreadable format that can only be accessed by authorized users.

The project titled **Secure File Encryption System** is designed to provide a reliable and efficient method for protecting files using password-based encryption techniques. The main objective of this system is to allow users to securely encrypt and decrypt files using a simple web-based interface while maintaining strong cryptographic security standards.

The system enables users to upload any type of file and encrypt it using a user-defined password. During the encryption process, the system generates a cryptographic key derived from the password using the **PBKDF2HMAC (Password-Based Key Derivation Function 2)** algorithm combined with a randomly generated salt. This method strengthens the password and protects it against brute-force attacks by performing multiple hashing iterations. The derived key is then used to perform encryption using the **Advanced Encryption Standard (AES-256)** algorithm operating in **Cipher Block Chaining (CBC)** mode.

The encrypted output is stored in a secure `.enc` file format that contains the encrypted data along with the salt and initialization vector required for decryption. When the user wants to retrieve the original file, the system uses the same password to regenerate the encryption key and decrypt the file securely.

The system is implemented using a modern **full-stack architecture**. The frontend interface is developed using **React (Vite)** and **TailwindCSS**, which provides a responsive and user-friendly dashboard for file uploading and encryption operations. The backend is implemented using the **Python Flask framework**, which handles file processing, cryptographic operations, and API communication between the frontend and backend systems. Secure cross-origin communication is enabled using **Flask-CORS**.

This project demonstrates the practical application of modern cryptographic techniques used in cybersecurity. It highlights how secure key generation, symmetric encryption, and password-based protection mechanisms can be integrated into a web application to protect sensitive files. The Secure File Encryption System can serve as a foundational model for building secure storage systems, data protection tools, and encrypted file sharing applications.

Contents

1 Introduction **4**
1.1 Objectives 5

2 Technologies Used **6**
2.1 Frontend Technologies 6
2.1.1 React (Vite Framework) 6
2.1.2 TailwindCSS 6
2.1.3 Axios 7
2.1.4 Lucide React Icons 7
2.1.5 HTML, CSS, and JavaScript 7
2.2 Backend Technologies 7
2.2.1 Python 3 7
2.2.2 Flask Framework 7
2.2.3 Cryptography Python Library 7
2.2.4 Flask-CORS 8

3 System Architecture **9**
3.1 System Architecture Diagram 9
3.2 Project Interface 9
3.3 Workflow of the System 9

4 Core Implementation Code **12**
4.1 Encryption and Decryption Implementation 12
4.2 Flask API Implementation 12

5 Implementation **14**
5.1 Encryption Process 14
5.2 Decryption Process 15
5.3 Security Considerations 15

6 Advantages **17**

7 Limitations **18**

8 Future Scope **19**

9 Conclusion **20**

Chapter 1

Introduction

In today's digital world, a vast amount of information is created, stored, and shared electronically. From personal documents and financial records to confidential organizational data, digital files have become an essential part of everyday life. However, the increasing use of digital technologies has also increased the risk of unauthorized access, data theft, and cyber attacks. Therefore, protecting sensitive information has become a critical requirement in modern computing systems.

Cybersecurity focuses on protecting systems, networks, and data from digital attacks. One of the most effective techniques used in cybersecurity to protect sensitive information is **encryption**. Encryption is the process of converting readable data (known as plaintext) into an unreadable format (called ciphertext) using mathematical algorithms. Only authorized users who possess the correct decryption key can convert the encrypted data back into its original readable form.

In many real-world applications such as secure file storage, cloud computing, online banking, and data transmission systems, encryption plays a vital role in ensuring confidentiality and data integrity. Without proper encryption mechanisms, sensitive files may be exposed to attackers, leading to privacy violations, financial loss, and misuse of confidential information.

The **Secure File Encryption System** developed in this project aims to provide a simple and secure method for protecting files using modern cryptographic techniques. The system allows users to upload files and encrypt them using a password-based encryption method. Once encrypted, the file is converted into a protected format that cannot be accessed without the correct password. When the user wants to retrieve the original file, the system decrypts the encrypted data using the same password that was used during encryption.

This project uses the **Advanced Encryption Standard (AES-256)** algorithm, which is one of the most secure and widely used symmetric encryption algorithms in modern cybersecurity systems. To enhance password security, the system uses the **PBKDF2 (Password-Based Key Derivation Function 2)** hashing algorithm combined with a random salt value. This technique strengthens the password by performing multiple hashing iterations, making it more resistant to brute-force and dictionary attacks.

The system is implemented using a **full-stack web application architecture**. The frontend interface is developed using **React** and **TailwindCSS**, which provides a clean and responsive user interface for uploading, encrypting, and decrypting files. The backend is developed using the **Python Flask framework**, which handles encryption operations, file processing, and communication with the frontend through API endpoints.

The Secure File Encryption System demonstrates how modern cryptographic techniques can be implemented in a real-world web application to ensure secure data storage and file protection. This project also helps students understand the practical implementation of encryption algorithms, key generation techniques, and secure data handling methods in cybersecurity applications.

1.1 Objectives

The main objectives of developing the Secure File Encryption System are as follows:

- To design and develop a secure system for encrypting and decrypting files.
- To implement the AES-256 symmetric encryption algorithm for strong data protection.
- To generate secure encryption keys using the PBKDF2 password-based hashing technique.
- To develop a modern and user-friendly web interface using React and TailwindCSS.
- To demonstrate practical applications of cryptography in cybersecurity systems.
- To provide a secure method for protecting sensitive digital files from unauthorized access.

Chapter 2

Technologies Used

The Secure File Encryption System is developed using a combination of modern frontend and backend technologies. These technologies work together to provide a secure, responsive, and efficient system for encrypting and decrypting files. The frontend is responsible for providing the user interface and handling user interactions, while the backend manages encryption, decryption, and secure file processing.

2.1 Frontend Technologies

The frontend of the system is responsible for creating an interactive and user-friendly interface through which users can upload files, enter passwords, and perform encryption or decryption operations.

2.1.1 React (Vite Framework)

React is a popular JavaScript library used for building modern and dynamic user interfaces. It allows developers to create reusable components and manage application state efficiently. In this project, React is used to design the main dashboard interface where users can upload files and perform encryption or decryption operations.

The project uses the **Vite framework** for React development. Vite is a fast development tool that provides quick project setup, instant server start, and optimized builds for production. It significantly improves the development experience by providing faster compilation and module updates.

2.1.2 TailwindCSS

TailwindCSS is a utility-first CSS framework used to design modern and responsive user interfaces. Instead of writing large CSS files, Tailwind provides predefined utility classes that can be directly used in HTML or JSX code to style components.

In this project, TailwindCSS is used to design the layout of the encryption and decryption panels, buttons, input fields, and other interface components. It helps create a clean, responsive, and visually appealing interface.

2.1.3 Axios

Axios is a JavaScript HTTP client used to send requests from the frontend to the backend server. In this project, Axios is used to send file data and passwords from the React frontend to the Flask backend through API endpoints. It supports asynchronous communication and simplifies handling of HTTP requests and responses.

2.1.4 Lucide React Icons

Lucide React is a modern icon library used to include lightweight and customizable icons in React applications. Icons are used in this project to enhance the visual design of the user interface, such as encryption buttons, file upload icons, and navigation elements.

2.1.5 HTML, CSS, and JavaScript

HTML (HyperText Markup Language) is used to structure the content of the web interface. CSS (Cascading Style Sheets) is used for styling and layout design, while JavaScript is used to add interactivity and dynamic behavior to the web application. Together, these core web technologies form the foundation of the frontend system.

2.2 Backend Technologies

The backend of the system is responsible for handling encryption, decryption, file processing, and secure communication between the server and the client.

2.2.1 Python 3

Python is a powerful and widely used programming language known for its simplicity and readability. It is commonly used in cybersecurity, data science, and backend web development. In this project, Python is used to implement the core logic of file encryption and decryption.

2.2.2 Flask Framework

Flask is a lightweight web framework for Python that allows developers to build web applications and APIs quickly. In this project, Flask is used to create API endpoints that receive files from the frontend, process encryption or decryption operations, and return the processed files to the user.

Flask is chosen because it is simple, flexible, and suitable for building small to medium-scale web applications.

2.2.3 Cryptography Python Library

The Cryptography library is a powerful Python package used to perform secure cryptographic operations. In this project, it is used to implement the AES-256 encryption algorithm and the PBKDF2 key derivation function. The library provides secure implementations of modern cryptographic standards and ensures that encryption operations follow best security practices.

2.2.4 Flask-CORS

Flask-CORS is an extension used to enable Cross-Origin Resource Sharing (CORS) in Flask applications. Since the frontend and backend may run on different ports or domains during development, CORS allows the frontend to securely communicate with the backend APIs without being blocked by browser security policies.

By combining these frontend and backend technologies, the Secure File Encryption System achieves a secure and efficient architecture that allows users to easily encrypt and decrypt files through a modern web interface.

Chapter 3

System Architecture

The Secure File Encryption System follows a **client-server architecture**. In this architecture, the frontend handles user interaction while the backend performs encryption and decryption operations. The system ensures secure communication between the client and the server using API requests.

The architecture consists of two major components:

- **Frontend (Client Side)** – Developed using React and TailwindCSS, it provides the user interface where users upload files and enter passwords.
- **Backend (Server Side)** – Developed using Python Flask, it handles file processing, encryption, and decryption using cryptographic algorithms.

3.1 System Architecture Diagram

The following diagram illustrates the overall architecture of the Secure File Encryption System.

3.2 Project Interface

The following figure shows the user interface of the Secure File Encryption System. The interface allows users to upload files, enter a password, and perform encryption or decryption operations through a simple dashboard.

3.3 Workflow of the System

The working process of the Secure File Encryption System can be explained through the following steps:

1. The user opens the web interface and selects a file for encryption or decryption.
2. The user enters a password which will be used for generating the encryption key.
3. The frontend sends the file and password to the backend server using an API request.
4. The backend processes the request and performs encryption or decryption using AES-256.

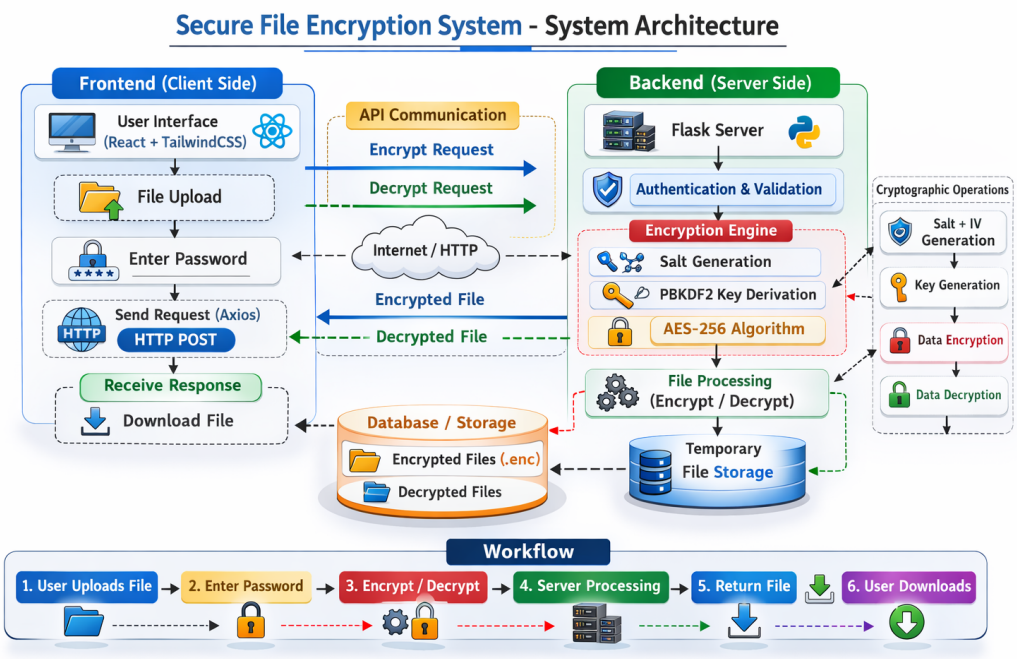

Figure 3.1: System Architecture of Secure File Encryption System

5. The processed file is returned to the frontend.
6. The user downloads the encrypted or decrypted file.

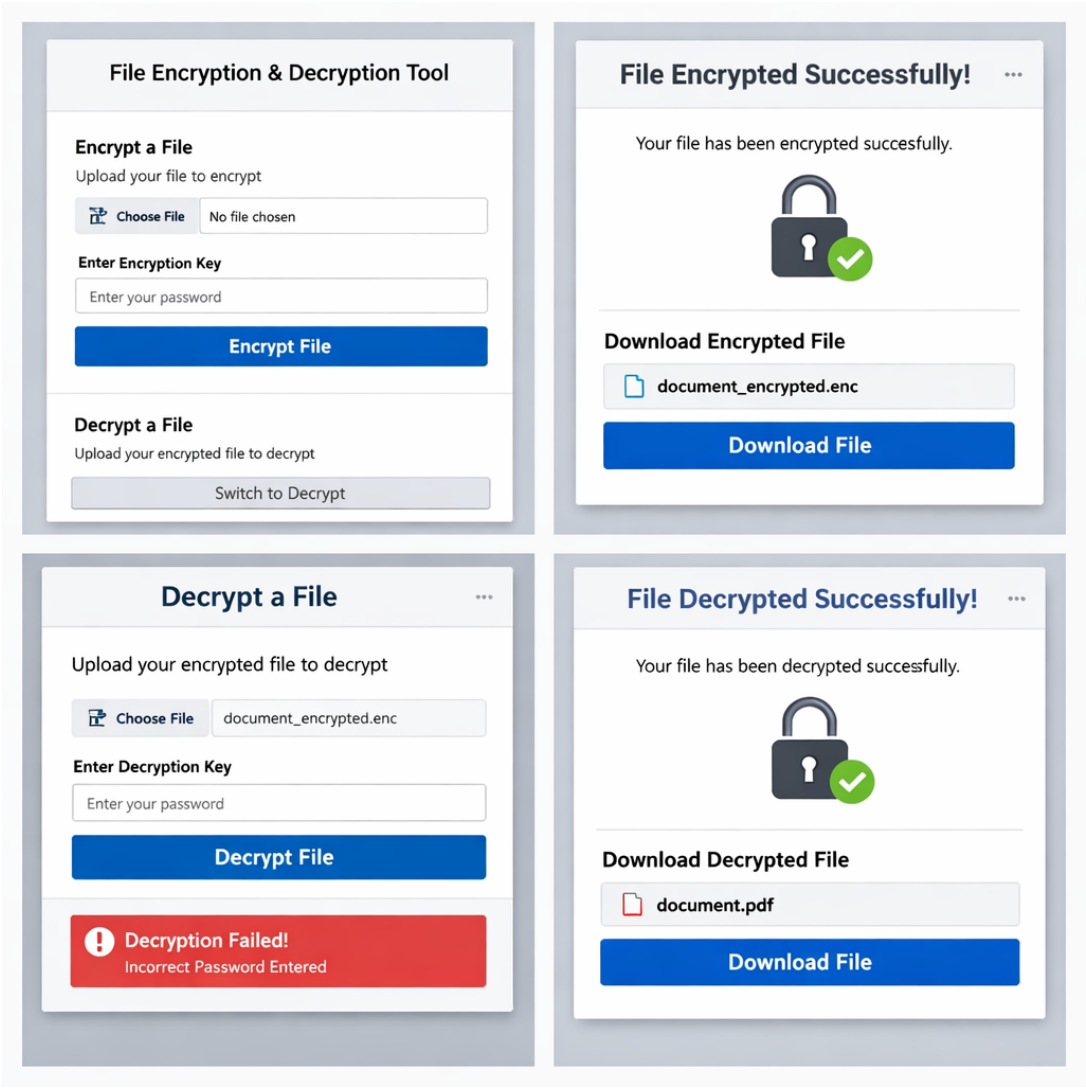

Figure 3.2: User Interface of Secure File Encryption System

Chapter 4

Core Implementation Code

This chapter presents the main implementation code used in the Secure File Encryption System. The following figures show the key parts of the backend encryption algorithm and API implementation used in the project.

4.1 Encryption and Decryption Implementation

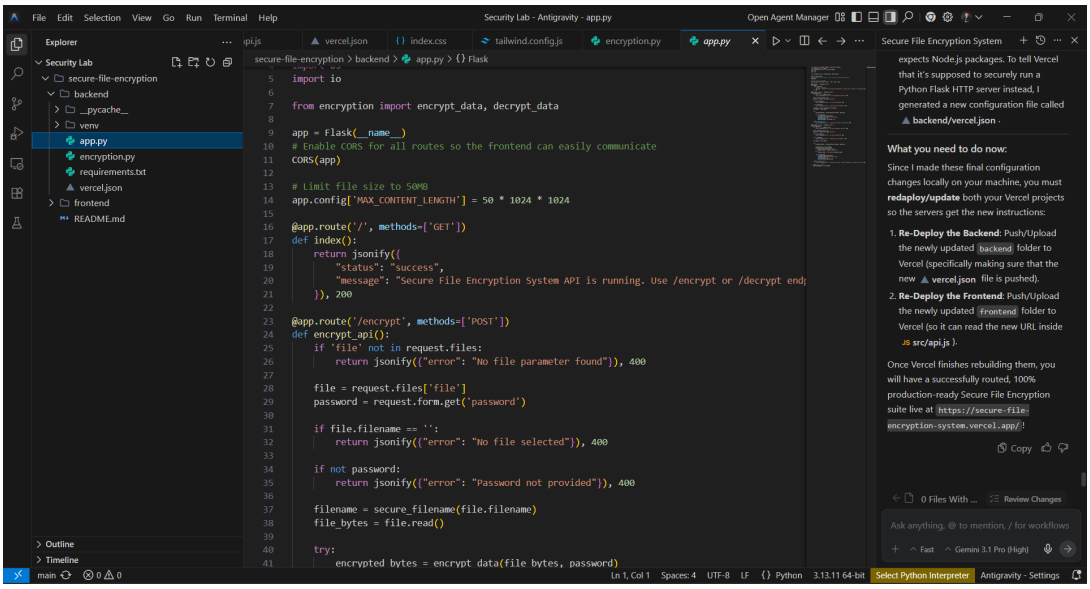

Figure 4.1: Main Encryption Algorithm Implementation

4.2 Flask API Implementation

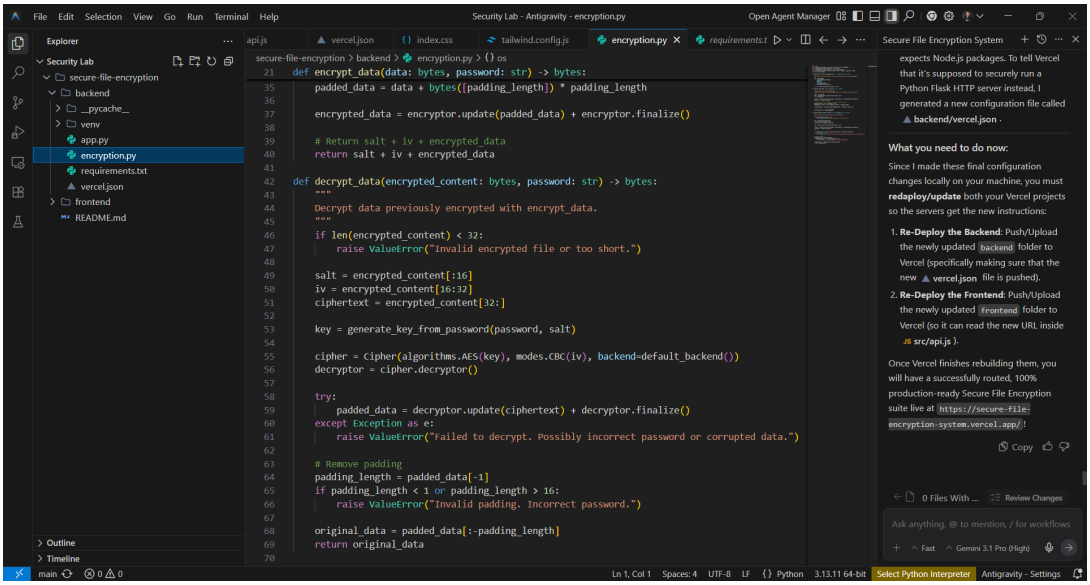

Figure 4.2: Flask Backend API for File Encryption and Decryption

Chapter 5

Implementation

The implementation of the Secure File Encryption System focuses on providing a secure and efficient method for encrypting and decrypting files using modern cryptographic techniques. The system is implemented using a full-stack architecture where the frontend is responsible for user interaction and the backend handles encryption and decryption operations.

The frontend interface allows users to upload files and provide a password for encryption or decryption. The backend server processes the file using cryptographic algorithms and returns the processed file to the user. The implementation primarily consists of two major processes: the **Encryption Process** and the **Decryption Process**.

5.1 Encryption Process

The encryption process converts the original file into a secure encrypted format so that unauthorized users cannot access its content. The system uses the AES-256 symmetric encryption algorithm along with the PBKDF2 key derivation function to generate a strong encryption key from the user-provided password.

The encryption process works as follows:

1. The user uploads a file through the web interface and enters a password that will be used to protect the file.

2. The frontend sends the file and password to the backend server using an HTTP POST request.

3. The backend generates a random **16-byte salt** value to strengthen the password and prevent dictionary attacks.

4. The password and salt are processed using the **PBKDF2HMAC key derivation algorithm**. This algorithm repeatedly hashes the password multiple times to generate a strong cryptographic key.

5. The output of the key derivation function produces a **256-bit encryption key** which is used for AES encryption.

6. The system then generates a random **Initialization Vector (IV)** which ensures that even identical files encrypted with the same password will produce different ciphertext.

7. The file data is encrypted using the **AES-256 algorithm in Cipher Block Chaining (CBC) mode**.
8. After encryption, the salt and initialization vector are attached to the encrypted data so that they can be used during the decryption process.
9. The final encrypted output is saved with the `.enc` file extension and returned to the user for download.

This encryption process ensures strong data protection because the encryption key is never directly stored and is always derived from the password provided by the user.

5.2 Decryption Process

The decryption process is responsible for restoring the encrypted file back to its original readable format. Only users who provide the correct password can successfully decrypt the file.

The decryption process works as follows:

1. The user uploads the previously encrypted `.enc` file through the web interface.
2. The user enters the password that was originally used during encryption.
3. The frontend sends the encrypted file and password to the backend server.
4. The backend extracts the first portion of the encrypted file, which contains the **salt** and the **initialization vector**.
5. Using the extracted salt and the user-provided password, the system regenerates the same encryption key using the **PBKDF2HMAC key derivation function**.
6. The encrypted data is then processed using the **AES-256-CBC decryption algorithm**.
7. The decrypted output is converted back into the original file format.
8. The restored file is returned to the user for download.

If the user provides an incorrect password, the system will fail to correctly regenerate the encryption key, and the file will not be decrypted successfully. This ensures that encrypted files remain protected from unauthorized access.

5.3 Security Considerations

Several security techniques are implemented in this system to ensure strong data protection:

- **AES-256 Encryption:** Provides strong symmetric encryption widely used in modern security systems.

- **PBKDF2 Key Derivation:** Strengthens passwords by performing multiple hashing iterations.
- **Random Salt Generation:** Prevents attackers from using precomputed hash tables such as rainbow tables.
- **Initialization Vector (IV):** Ensures that identical files encrypted with the same password produce different encrypted outputs.
- **Temporary File Processing:** Files are processed temporarily and not permanently stored on the server, improving security and privacy.

These security measures ensure that the Secure File Encryption System provides reliable protection for sensitive digital files.

Chapter 6

Advantages

The Secure File Encryption System provides several benefits in terms of data protection, usability, and security. The main advantages of the system are described below:

- **Strong Encryption using AES-256:** The system uses the Advanced Encryption Standard (AES-256) algorithm which is one of the most secure symmetric encryption algorithms available today. It is widely used in government, banking, and cybersecurity applications to protect sensitive information.
- **Secure Key Generation:** The encryption key is generated using the PBKDF2 (Password-Based Key Derivation Function 2) algorithm combined with a randomly generated salt. This technique strengthens the password by performing multiple hashing iterations and protects the system from brute-force and dictionary attacks.
- **Password-Based Security:** The system ensures that files can only be decrypted using the correct password provided during encryption. This provides an additional layer of protection and ensures that unauthorized users cannot access encrypted data.
- **User-Friendly Interface:** The project provides a simple and intuitive web interface built using React and TailwindCSS. Users can easily upload files, enter passwords, and perform encryption or decryption operations without requiring advanced technical knowledge.
- **Support for Multiple File Types:** The system allows users to encrypt various types of files including documents, images, videos, and other digital files. This flexibility makes the system useful for protecting different kinds of sensitive data.
- **Secure File Handling:** Files are processed temporarily on the backend server and are not permanently stored in the system. This improves data privacy and reduces the risk of sensitive information being stored on the server.
- **Platform Independent Access:** Since the system is developed as a web application, it can be accessed from different devices and operating systems through a web browser.

Chapter 7

Limitations

Although the Secure File Encryption System provides strong data protection, there are some limitations that may affect its usage in certain situations.

- **Password Dependency:** The system relies completely on the password provided by the user. If the user forgets the password, it becomes impossible to recover or decrypt the encrypted file because the encryption key cannot be regenerated.
- **Processing Time for Large Files:** Encryption and decryption operations may take more time when very large files are processed. This can slightly affect performance depending on the system resources and network speed.
- **Backend Server Requirement:** The system requires an active backend server to perform encryption and decryption operations. If the server is unavailable or offline, the system cannot process files.
- **Single User Operation:** The current version of the system is designed primarily for individual users and does not support multi-user account management or authentication features.
- **Limited File Management Features:** The system focuses mainly on encryption and decryption and does not include advanced file management features such as file history, storage management, or file sharing.

Chapter 8

Future Scope

The Secure File Encryption System can be further improved by adding several advanced features and improvements in the future. Some possible future enhancements are listed below:

- **Cloud Storage Integration:** Future versions of the system can integrate with cloud storage platforms such as Google Drive, AWS, or Dropbox to securely store encrypted files.
- **Multi-Factor Authentication:** Additional authentication mechanisms such as OTP verification or biometric authentication can be implemented to provide stronger security.
- **Secure File Sharing:** The system can be extended to allow users to share encrypted files through secure links while maintaining access control.
- **Drag-and-Drop File Upload:** Improving the user interface with drag-and-drop functionality can make file uploading faster and more convenient.
- **Mobile Application Support:** A mobile application version of the system can be developed so users can encrypt and decrypt files directly from their smartphones.
- **User Authentication System:** Future improvements may include user login, account management, and role-based access control to support multiple users securely.
- **Improved Performance Optimization:** The system can be optimized further to handle large files more efficiently using streaming encryption techniques.

Chapter 9

Conclusion

The Secure File Encryption System successfully demonstrates how modern cryptographic techniques can be applied to protect sensitive digital information. With the increasing amount of data being stored and shared electronically, ensuring data security has become a critical requirement in today's digital world. This project addresses that challenge by providing a secure and user-friendly system for encrypting and decrypting files.

The system uses the **AES-256 symmetric encryption algorithm**, which is widely recognized as one of the most secure encryption standards used in modern cybersecurity applications. By combining AES encryption with the **PBKDF2 password-based key derivation function**, the system strengthens user passwords and generates secure encryption keys. The use of random **salt values** and **initialization vectors (IV)** further enhances the security of the encryption process.

The project is implemented using a **full-stack web architecture**. The frontend, developed using React and TailwindCSS, provides a clean and responsive interface that allows users to easily upload files and perform encryption or decryption operations. The backend, implemented using Python Flask and the Cryptography library, performs secure cryptographic processing and ensures safe file handling.

One of the key advantages of this system is its simplicity and accessibility. Users can securely protect files without requiring advanced technical knowledge. At the same time, the system incorporates strong cryptographic practices that ensure data confidentiality and protection against unauthorized access.

Through this project, important cybersecurity concepts such as symmetric encryption, key derivation, secure password handling, and client-server architecture have been practically implemented and demonstrated. The Secure File Encryption System serves as a practical example of how encryption technologies can be integrated into real-world applications to protect digital data.

In conclusion, the project successfully achieves its objective of developing a secure file encryption platform while also demonstrating the practical application of modern web development technologies and cryptographic security mechanisms. With further improvements and additional features, this system can evolve into a more advanced secure file storage and sharing solution for real-world environments.